

```

----- administrar.html
-----
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Banco Escolar - Administración General</title>
  <link rel="stylesheet" href="css/cajero.css">
  <style>
    .admin-grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
      gap: 20px;
      margin-top: 20px;
    }

    .badge-status {
      padding: 3px 8px;
      border-radius: 4px;
      font-size: 0.8rem;
      font-weight: bold;
    }

    .badge-ok {
      background: #c6f6d5;
      color: #22543d;
    }

    .badge-off {
      background: #fed7d7;
      color: #742a2a;
    }
  </style>
</head>

<body>

  <div id="navbar-container"></div>

  <main class="dashboard-container" style="max-width: 1200px; margin: 0 auto;">
    <h2>🌀 Panel de Administración General</h2>
    <p class="text-subtitle">Gestión de accesos, permisos, terminales POSNET y
parametrización.</p>

    <div class="admin-grid">

      <!-- SECCIÓN 1: GESTIÓN DE USUARIOS BANCO (Cajeros y Admins) -->
      <section class="card">

```

```

<h3>👤 Usuarios del Sistema</h3>
<p class="text-subtitle">Crear cajeros/admins y gestionar passwords</p>

<form id="formUsuarioBanco" style="margin-bottom: 20px;">
  <input type="hidden" id="usrId">

  <div class="form-group">
    <label for="txtUsrUsuario">Usuario / Login</label>
    <input type="text" id="txtUsrUsuario" placeholder="Ej: cajero_turno1"
required>
  </div>

  <div class="form-group">
    <label for="txtUsrNombre">Nombre Completo</label>
    <input type="text" id="txtUsrNombre" placeholder="Ej: María López"
required>
  </div>

  <div class="form-group">
    <label for="txtUsrPass">Contraseña / Password</label>
    <input type="password" id="txtUsrPass" placeholder="....." required>
  </div>

  <div class="form-group">
    <label for="selUsrRol">Rol de Sistema</label>
    <select id="selUsrRol">
      <option value="CAJERO">CAJERO</option>
      <option value="ADMIN">ADMINISTRADOR</option>
    </select>
  </div>

  <div class="form-group" style="display: flex; gap: 15px; align-items:
center; margin-top: 10px;">
    <label><input type="checkbox" id="chkPuedeRetirar"> ¿Permite Retiro de
Efectivo?</label>
  </div>

  <div style="display: flex; gap: 10px;">
    <button type="submit" id="btnGuardarUsr" class="btn btn-primary"
style="flex: 1;">Crear Usuario</button>
    <button type="button" id="btnCancelarUsr" class="btn"
style="display: none; background: #718096; color:
white;">Cancelar</button>
  </div>
</form>

<div style="overflow-x: auto;">
  <table class="tabla-datos" style="width: 100%;">
    <thead>
      <tr>

```

```

        <th>Usuario</th>
        <th>Rol</th>
        <th>Retiro</th>
        <th>Estado</th>
        <th>Acciones</th>
    </tr>
</thead>
<tbody id="tblUsuariosBanco"></tbody>
</table>
</div>
</section>

<!-- SECCIÓN 2: CONTROL DE TERMINALES POSNET -->
<section class="card">
    <h3> POSNETs / Terminales de Cobro</h3>
    <p class="text-subtitle">Alta y activación de puntos de cobro</p>

    <form id="formPosnet" style="margin-bottom: 20px;">
        <div class="form-group">
            <label for="txtPosnetNombre">Nombre del Stand / Puesto</label>
            <input type="text" id="txtPosnetNombre" placeholder="Ej: Stand Buffet
5to Año" required>
        </div>

        <!-- Campos obligatorios de credenciales para la terminal POSNET -->
        <div class="form-group">
            <label for="txtPosnetUsuario">Usuario de Acceso POSNET</label>
            <input type="text" id="txtPosnetUsuario" placeholder="Ej:
posnet_buffet" required>
        </div>

        <div class="form-group">
            <label for="txtPosnetPass">Contraseña POSNET</label>
            <input type="password" id="txtPosnetPass" placeholder="....."
required>
        </div>

        <div class="form-group">
            <label for="txtPosnetSerie">Nº de Serie / Dispositivo</label>
            <input type="text" id="txtPosnetSerie" placeholder="Ej: POS-001">
        </div>

        <button type="submit" class="btn btn-primary" style="width:
100%;">Registrar Terminal</button>
    </form>

    <div style="overflow-x: auto;">
        <table class="tabla-datos" style="width: 100%;">
            <thead>
                <tr>

```

```

        <th>Stand / Usuario</th>
        <th>Nº Serie</th>
        <th>Acumulado</th>
        <th>Estado</th>
        <th>Acción</th>
    </tr>
</thead>
<tbody id="tblPosnets"></tbody>
</table>
</div>
</section>

<!-- SECCIÓN 3: CRUD DE CURSOS / DIVISIONES -->
<section class="card">
    <h3>🏠 Cursos y Divisiones</h3>
    <p class="text-subtitle">Precarga de años/cursos para el alta de
alumnos</p>

    <form id="formCurso" style="margin-bottom: 20px; display: flex; gap:
10px;">
        <input type="text" id="txtNombreCurso" placeholder="Ej: 6to 1ra
Informática" style="flex: 1;" required>
        <button type="submit" class="btn btn-primary">Agregar</button>
    </form>

    <div style="overflow-x: auto;">
        <table class="tabla-datos" style="width: 100%;">
            <thead>
                <tr>
                    <th>Nombre del Curso</th>
                    <th style="text-align: right;">Acción</th>
                </tr>
            </thead>
            <tbody id="tblCursos"></tbody>
        </table>
    </div>
</section>

</div>
</main>

<script src="js/navbar.js"></script>
<script src="js/administrar.js"></script>
</body>

</html>

```

```

----- administrar.js
-----
// Verificar permisos de Administrador

```

```

const usuarioSesion = JSON.parse(localStorage.getItem('usuarioBanco')) ||
JSON.parse(sessionStorage.getItem('session')) || {};
if ((usuarioSesion.rol || '').toUpperCase() !== 'ADMIN') {
  alert("Acceso denegado. Se requieren permisos de Administrador.");
  window.location.href = 'cajero.html';
}

document.addEventListener('DOMContentLoaded', async () => {
  await cargarUsuariosBanco();
  await cargarPosnets();
  await cargarCursos();

  // Eventos de Formularios
  document.getElementById('formUsuarioBanco').addEventListener('submit',
guardarUsuarioBanco);
  document.getElementById('formPosnet').addEventListener('submit', guardarPosnet);
  document.getElementById('formCurso').addEventListener('submit', guardarCurso);
  document.getElementById('btnCancelarUsr').addEventListener('click',
resetFormUsuario);
});

// =====
// 1. GESTIÓN DE USUARIOS (Cajeros / Admins)
// =====
async function cargarUsuariosBanco() {
  const tbody = document.getElementById('tblUsuariosBanco');
  const client = window._supabase || supabase;

  try {
    const { data, error } = await
client.from('usuarios_banco').select('*').order('usuario');
    if (error) throw error;

    if (!data || data.length === 0) {
      tbody.innerHTML = `<tr><td colspan="5" style="text-align:center;">No hay
usuarios.</td></tr>`;
      return;
    }

    tbody.innerHTML = data.map(u => `
      <tr>
        <td><b>${u.usuario}</b><br><small>${u.nombre || ''}</small></td>
        <td>${u.rol}</td>
        <td>${u.puede_retirar ? '☑ Sí' : '✗ No'}</td>
        <td><span class="badge-status ${u.activo ? 'badge-ok' :
'badge-off'}">${u.activo ? 'Activo' : 'Inactivo'}</span></td>
        <td>
          <button onclick="editarUsuario('${u.id}', '${u.usuario}', '${u.nombre}',
'${u.rol}', ${u.puede_retirar})" class="btn" style="padding: 2px 6px; font-size:
0.8rem; background: #3182ce; color: white;">✎ Edit</button>
        </td>
      </tr>
    `);
  } catch (error) {
    console.error('Error al cargar usuarios:', error);
  }
}

```

```

        <button onclick="toggleEstadoUsuario('${u.id}', ${!u.activo})"
class="btn" style="padding: 2px 6px; font-size: 0.8rem; background: ${u.activo ?
'#e53e3e' : '#38a169'}; color: white;">${u.activo ? 'Desactivar' :
'Activar'}</button>
        </td>
    </tr>
    `).join('');
} catch (err) {
    console.error(err);
}
}
}

```

```

async function guardarUsuarioBanco(e) {
    e.preventDefault();
    const client = window._supabase || supabase;

    const id = document.getElementById('usrId').value;
    const usuario = document.getElementById('txtUsrUsuario').value.trim();
    const nombre = document.getElementById('txtUsrNombre').value.trim();
    const passwordHash = document.getElementById('txtUsrPass').value.trim();
    const rol = document.getElementById('selUsrRol').value;
    const puedeRetirar = document.getElementById('chkPuedeRetirar').checked;

    const payload = {
        usuario,
        nombre,
        rol,
        puede_retirar: puedeRetirar,
        activo: true
    };

    if (passwordHash) {
        payload.password_hash = passwordHash;
    }

    try {
        let error;
        if (id) {
            const idNumerico = parseInt(id, 10);
            ({ error } = await client.from('usuarios_banco').update(payload).eq('id',
idNumerico));
        } else {
            ({ error } = await client.from('usuarios_banco').insert([payload]));
        }

        if (error) throw error;

        alert(id ? "Usuario actualizado correctamente." : "Usuario creado con éxito.");
        resetFormUsuario();
        await cargarUsuariosBanco();
    }
}

```

```

    } catch (err) {
        alert("Error al guardar usuario: " + err.message);
    }
}

function editarUsuario(id, usuario, nombre, rol, puedeRetirar) {
    document.getElementById('usrId').value = id;
    document.getElementById('txtUsrUsuario').value = usuario;
    document.getElementById('txtUsrNombre').value = nombre;
    document.getElementById('txtUsrPass').value = '';
    document.getElementById('txtUsrPass').placeholder = '(Dejar en blanco para mantener)';
    document.getElementById('txtUsrPass').required = false;
    document.getElementById('selUsrRol').value = rol;
    document.getElementById('chkPuedeRetirar').checked = puedeRetirar;

    document.getElementById('btnGuardarUsr').innerText = "Actualizar Usuario";
    document.getElementById('btnCancelarUsr').style.display = "inline-block";
}

function resetFormUsuario() {
    document.getElementById('formUsuarioBanco').reset();
    document.getElementById('usrId').value = '';
    document.getElementById('txtUsrPass').required = true;
    document.getElementById('txtUsrPass').placeholder = '.....';
    document.getElementById('btnGuardarUsr').innerText = "Crear Usuario";
    document.getElementById('btnCancelarUsr').style.display = "none";
}

async function toggleEstadoUsuario(id, nuevoEstado) {
    const client = window._supabase || supabase;
    const idNumerico = parseInt(id, 10);

    const { error } = await client
        .from('usuarios_banco')
        .update({ activo: nuevoEstado })
        .eq('id', idNumerico);

    if (error) {
        console.error("Error de Supabase:", error);
        alert("Error al actualizar: " + error.message);
    } else {
        await cargarUsuariosBanco();
    }
}

// =====
// 2. GESTIÓN DE POSNETS (CORREGIDO)
// =====
// =====

```

```

// 2. GESTIÓN DE POSNETS (REVISADO)
// =====
async function cargarPosnets() {
  const tbody = document.getElementById('tblPosnets');
  if (!tbody) return;

  const client = window._supabase || supabase;

  try {
    const { data, error } = await client
      .from('posnets')
      .select('*')
      .order('nombre_posnet', { ascending: true });

    if (error) {
      console.error("Error de Supabase al consultar posnets:", error);
      tbody.innerHTML = `<tr><td colspan="5" style="text-align:center; color:
red;">Error al cargar datos: ${error.message}</td></tr>`;
      return;
    }

    if (!data || data.length === 0) {
      tbody.innerHTML = `<tr><td colspan="5" style="text-align:center;">No hay
POSNETs configurados.</td></tr>`;
      return;
    }

    tbody.innerHTML = data.map(p => {
      const monto = Number(p.monto_acumulado || 0).toFixed(2);
      const estadoTexto = p.habilitado ? 'Activo' : 'Inactivo';
      const badgeClase = p.habilitado ? 'badge-ok' : 'badge-off';
      const botonTexto = p.habilitado ? 'Baja' : 'Alta';
      const botonColor = p.habilitado ? '#e53e3e' : '#38a169';

      return `
        <tr>
          <td>
            <b>${p.nombre_posnet || 'Sin nombre'}</b><br>
            <small style="color: #718096;">Usr: ${p.usuario || '-'}</small>
          </td>
          <td>${p.device_id_autorizado || '-'}</td>
          <td>${p.monto}</td>
          <td><span class="badge-status ${badgeClase}">${estadoTexto}</span></td>
          <td>
            <button onclick="toggleEstadoPosnet('${p.id}', ${!p.habilitado})"
class="btn" style="padding: 2px 6px; font-size: 0.8rem; background: ${botonColor};
color: white;">
              ${botonTexto}
            </button>
          </td>
        </tr>
      `;
    });
  }
}

```



```

        </tr>
    `;
    }).join('');

    } catch (err) {
        console.error("Error inesperado en cargarPosnets:", err);
        tbody.innerHTML = `<tr><td colspan="5" style="text-align:center; color:
red;">Error inesperado al cargar lista.</td></tr>`;
    }
}

async function guardarPosnet(e) {
    e.preventDefault();
    const client = window._supabase || supabase;

    const nombre = document.getElementById('txtPosnetNombre').value.trim();
    const usuario = document.getElementById('txtPosnetUsuario').value.trim();
    const pass = document.getElementById('txtPosnetPass').value.trim();
    const serie = document.getElementById('txtPosnetSerie').value.trim();

    const payload = {
        nombre_posnet: nombre,
        usuario: usuario,
        password: pass,
        device_id_authorized: serie || null,
        habilitado: true,
        monto_acumulado: 0.00,
        cant_transacciones: 0
    };

    const { error } = await client.from('posnets').insert([payload]);

    if (error) {
        alert("Error al registrar POSNET: " + error.message);
        console.error("Detalle del error:", error);
    } else {
        alert("¡POSNET registrado con éxito!");
        document.getElementById('formPosnet').reset();
        await cargarPosnets(); // Recarga la tabla de inmediato
    }
}

async function toggleEstadoPosnet(idUuid, nuevoEstado) {
    const client = window._supabase || supabase;

    // ⚠ Importante: 'id' en posnets es UUID, NO se convierte con parseInt
    const { error } = await client.from('posnets').update({ habilitado: nuevoEstado
}).eq('id', idUuid);
    if (error) {
        alert("Error al cambiar estado del POSNET: " + error.message);
    }
}

```

```

    } else {
      await cargarPosnets();
    }
  }

// =====
// 3. CRUD DE CURSOS
// =====
async function cargarCursos() {
  const tbody = document.getElementById('tblCursos');
  const client = window._supabase || supabase;

  const { data } = await client.from('cursos').select('*').order('nombre');
  if (!data || data.length === 0) {
    tbody.innerHTML = `<tr><td colspan="2" style="text-align:center;">No hay cursos
cargados.</td></tr>`;
    return;
  }

  tbody.innerHTML = data.map(c => `
    <tr>
      <td><b>${c.nombre}</b></td>
      <td style="text-align: right;">
        <button onclick="eliminarCurso('${c.id}')" class="btn" style="padding: 2px
6px; font-size: 0.8rem; background: #e53e3e; color: white;">🗑 Eliminar</button>
      </td>
    </tr>
  `).join('');
}

async function guardarCurso(e) {
  e.preventDefault();
  const client = window._supabase || supabase;
  const nombre = document.getElementById('txtNombreCurso').value.trim();

  const { error } = await client.from('cursos').insert([ { nombre } ]);
  if (error) {
    alert("Error al guardar curso: " + error.message);
  } else {
    document.getElementById('formCurso').reset();
    await cargarCursos();
  }
}

async function eliminarCurso(id) {
  if (!confirm("¿Eliminar este curso?")) return;
  const client = window._supabase || supabase;
  const idNumerico = parseInt(id, 10);

  await client.from('cursos').delete().eq('id', idNumerico);
}

```

```
    await cargarCursos();  
}
```